steady.

Definition 1. *(Dropout Stability) A solution θ is ϵ-**dropout stable** if for all i such that $1 \leq i < d$, there exists a subset of at most $\lfloor h_j/2 \rfloor$ hidden units in each of the layers j from i through $d-1$ such that after rescaling the outputs of these hidden units (or equivalently, the corresponding rows and/or columns of the relevant weight matrices) by some factor r[1] and setting the outputs of the remaining units to zero, we obtain a parameter θ_i such that $L(f_{\theta_i}) \leq L(f_\theta) + \epsilon$.*

Intuitively, if a solution is ϵ-dropout stable then it is essentially only using half of the network's capacity. We show that such solutions are connected:

Theorem 1. *Let θ^A and θ^B be two ϵ-dropout stable solutions. Then there exists a path in parameter space $\pi : [0,1] \to \Theta$ between θ^A and θ^B such that $L(f_{\pi(t)}) \leq \max\{L(f_{\theta^A}), L(f_{\theta^B})\} + \epsilon$ for $0 \leq t \leq 1$. In other words, letting $\mathcal{C}$ be the set of solutions that are ϵ-dropout stable, a ReLU network has the ϵ-mode connectivity property with respect to $\mathcal{C}$.*

Our path construction in Theorem 1 consists of two key steps. First we show that we can rescale at least half the hidden units in both θ^A and θ^B to zero via continuous paths of low loss, thus obtaining two parameters θ_1^A and θ_1^B satisfying the criteria in Definition 1.

Lemma 1. *Let θ be an ϵ-dropout stable solution and let θ_i be specified as in Definition 1 for $1 \leq i < d$. Then there exists a path in parameter space $\pi : [0,1] \to \Theta$ between θ and θ_1 passing through each θ_i such that $L(f_{\pi(t)}) \leq L(f_\theta) + \epsilon$ for $0 \leq t \leq 1$.*

Though naïvely one might expect to be able to directly connect the weights of θ and θ_1 via interpolation, such a path may incur high loss as the loss function is not convex over Θ. In our proof of Lemma 1, we rely on a much more careful construction. The construction uses two types of steps: (a) interpolate between two weights in the top layer (the loss is convex in the top layer weights); (b) if a set of neurons already have their output weights set to zero, then we can change their input weights arbitrarily. See Figure 1 for an example path for a 3-layer network. Here we have separated the weight matrices into equally sized blocks: $A_3 = \left[\begin{array}{c|c} L_3 & R_3 \end{array}\right]$, $A_2 = \left[\begin{array}{c|c} L_2 & C_2 \\ \hline D_2 & R_2 \end{array}\right]$ and $A_1 = \left[\begin{array}{c} L_1 \\ \hline B_1 \end{array}\right]$. The path consists of 6 steps alternating between type (a) and type (b). Note that for all the type (a) steps, we only update the top layer weights; for all the type (b) steps, we only change rows of a weight matrix (inputs to neurons) if the corresponding columns in the previous matrix (outputs of neurons) are already 0. In Section A we show how such a path can be generalized to any number of layers.

We then show that we can permute the hidden units of θ_1^A such that its non-zero units do not intersect with those of θ_1^B, thus allowing us two interpolate between these two parameters. This is formalized in the following lemma and the proof is deferred to supplementary material.

Lemma 2. *Let θ and θ' be two solutions such that at least $\lceil h_i/2 \rceil$ of the units in the ith hidden layer have been set to zero in both. Then there exists a path in parameter space $\pi : [0,1] \to \Theta$ between θ and θ' with 8 line segments such that $L(f_{\pi(t)}) \leq \max\{L(f_\theta), L(f_{\theta'})\}$.*

Theorem 1 follows immediately from Lemma 1 and Lemma 2, as one can first connect θ^A to its dropout version θ_1^A using Lemma 1, then connect θ_1^A to dropout version θ_1^B of θ^B using Lemma 2, and finally connect θ_1^B to θ^B using Lemma 1 again.

Finally, our results can be generalized to convolutional networks if we do *channel-wise* dropout (Tompson et al., 2015; Keshari et al., 2018).

Remark 1. *For convolutional networks, a* channel-wise *dropout will randomly set entire channels to 0 and rescale the remaining channels using an appropriate factor. Theorem 1 can be extended to work with channel-wise dropout on convolutional networks.*

[1] Note our results will also work if r is allowed to vary for each layer.

$$
\begin{array}{lccccc}
 & A_3 & A_2 & A_1 & \\
(1) & \left[\begin{array}{c|c} L_3 & R_3 \end{array}\right] & \left[\begin{array}{c|c} L_2 & C_2 \\ \hline D_2 & R_2 \end{array}\right] & \left[\begin{array}{c} L_1 \\ \hline B_1 \end{array}\right] & \\
(2) & \left[\begin{array}{c|c} rL_3 & 0 \end{array}\right] & \left[\begin{array}{c|c} L_2 & C_2 \\ \hline D_2 & R_2 \end{array}\right] & \left[\begin{array}{c} L_1 \\ \hline B_1 \end{array}\right] & (a) \\
(3) & \left[\begin{array}{c|c} rL_3 & 0 \end{array}\right] & \left[\begin{array}{c|c} L_2 & C_2 \\ \hline rL_2 & 0 \end{array}\right] & \left[\begin{array}{c} L_1 \\ \hline B_1 \end{array}\right] & (b) \\
(4) & \left[\begin{array}{c|c} 0 & rL_3 \end{array}\right] & \left[\begin{array}{c|c} L_2 & C_2 \\ \hline rL_2 & 0 \end{array}\right] & \left[\begin{array}{c} L_1 \\ \hline B_1 \end{array}\right] & (a) \\
(5) & \left[\begin{array}{c|c} 0 & rL_3 \end{array}\right] & \left[\begin{array}{c|c} rL_2 & 0 \\ \hline rL_2 & 0 \end{array}\right] & \left[\begin{array}{c} L_1 \\ \hline B_1 \end{array}\right] & (b) \\
(6) & \left[\begin{array}{c|c} rL_3 & 0 \end{array}\right] & \left[\begin{array}{c|c} rL_2 & 0 \\ \hline rL_2 & 0 \end{array}\right] & \left[\begin{array}{c} L_1 \\ \hline B_1 \end{array}\right] & (a) \\
(7) & \left[\begin{array}{c|c} rL_3 & 0 \end{array}\right] & \left[\begin{array}{c|c} rL_2 & 0 \\ \hline 0 & 0 \end{array}\right] & \left[\begin{array}{c} L_1 \\ \hline 0 \end{array}\right] & (b)
\end{array}
$$

Figure 1: Example path, 6 line segments from a 3-layer network to its dropout version. Red denotes weights that have changed between steps while green denotes the zeroed weights that allow us to make these changes without affecting our output.

4 Connectivity via noise stability

In this section, we relate mode connectivity to another notion of robustness for neural networks—noise stability. It has been observed (Morcos et al., 2018) that neural networks often perform as well even if a small amount of noise is injected into the hidden layers. This was formalized in (Arora et al., 2018), where the authors showed that noise stable networks tend to generalize well. In this section we use a very similar notion of noise stability, and show that all noise stable solutions can be connected as long as the network is sufficiently overparametrized.

We begin in Section 4.1 by restating the definitions of noise stability in (Arora et al., 2018) and also highlighting the key differences in our definitions. In Section 6 we verify these assumptions in practice. In Section 4.2, we first prove that noise stability implies dropout stability (meaning Theorem 1 applies) and then show that it is in fact possible to connect noise stable neural networks via even simpler paths than mere dropout stable networks.

4.1 Noise stability

First we introduce some additional notations and assumptions. In this section, we consider a finite and fixed training set S. For a network parameter θ, the empirical loss function is $L(\theta) = \frac{1}{|S|}\sum_{(x,y)\in S} l(y, f(x))$. Here the loss function $l(y, \hat{y})$ is assumed to be β-Lipschitz in $\hat{y}$: for any $\hat{y}, \hat{y}' \in \mathbb{R}^{h_d}$ and any $y \in \mathbb{R}^{h_d}$, we have $|l(y, \hat{y}) - l(y, \hat{y}')| \leq \beta\|\hat{y} - \hat{y}'\|$. Note that the standard cross entropy loss over the softmax function is $\sqrt{2}$-Lipschitz.

For any two layers $i \leq j$, let $M^{i,j}$ be the operator for the composition of these layers, such that $x^j = M^{i,j}(x^i)$. Let $J^{i,j}_{x^i}$ be the Jacobian of $M^{i,j}$ at input x^i. Since the activation functions are ReLU's, we know $M^{i,j}(x^i) = J^{i,j}_{x^i} x^i$.

Arora et al. (2018) used several quantities to define noise stability. We state the definitions of these quantities below.

Definition 2 (Noise Stability Quantities)**.** *Given a sample set S, the **layer cushion** of layer i is defined as* $\mu_i := \min_{x\in S} \frac{\|A_i\phi(x^{i-1})\|}{\|A_i\|_F\|\phi(x^{i-1})\|}$.

*For any two layers $i \le j$, the **interlayer cushion** $\mu_{i,j}$ is defined as* $\mu_{i,j} = \min_{x\in S} \frac{\|J^{i,j}_{x^i}x^i\|}{\|J^{i,j}_{x^i}\|\|x^i\|}$.

*Furthermore, for any layer i the **minimal interlayer cushion** is defined as*[2] $\mu_{i\to} = \min_{i\le j\le d} \mu_{i,j}$.

*The **activation contraction** c is defined as* $c = \max_{x\in S,\ 1\le i\le d-1} \frac{\|x^i\|}{\|\phi(x^i)\|}$.

Intuitively, these quantities measures the stability of the network's output to noise for both a single layer and across multiple layers. Note that the definition of the *interlayer cushion* is slightly different from the original definition in (Arora et al., 2018). Specifically, in the denominator of our definition of interlayer cushion, we replace the Frobenius norm of $J^{i,j}_{x^i}$ by its spectral norm. In the original definition, the interlayer cushion is at most $1/\sqrt{h_i}$, simply because $J^{i,i}_{x^i} = I_{h_i}$ and $\mu_{i,i} = 1/\sqrt{h_i}$. With this new definition, the interlayer cushion need not depend on the layer width h_i.

The final quantity of interest is interlayer smoothness, which measures how close the network's behavior is to its linear approximation under noise. Our focus here is on the noise generated by the dropout procedure (Algorithm 1). Let $\theta = \{A_1, A_2, ..., A_d\}$ be weights of the original network, and let $\theta^i = \{A_1, \hat{A}_2, \dots, \hat{A}_i, A_{i+1}, \dots, A_d\}$ be the result of applying Algorithm 1 to weight matrices from layer 2 to layer i.[3] For any input x, let $\hat{x}^i_i(t)$ and $\hat{x}^i_{i-1}(t)$ be the vector before activation at layer i using parameters $\theta t + \theta^i(1-t)$ and $\theta t + \theta^{i-1}(1-t)$ respectively.

Definition 3 (Interlayer Smoothness)**.** *Given the scenario above, define **interlayer smoothness** ρ to be the largest number such that with probability at least $1/2$ over the randomness in Algorithm 1 for any two layers i, j satisfying for every $2 \le i \le j \le d$, $x \in S$, and $0 \le t \le 1$*

$$\|M^{i,j}(\hat{x}^i_i(t)) - J^{i,j}_{x^i}(\hat{x}^i_i(t))\| \le \frac{\|\hat{x}^i_i(t) - x^i\|\|x^j\|}{\rho\|x^i\|},$$
$$\|M^{i,j}(\hat{x}^i_{i-1}(t)) - J^{i,j}_{x^i}(\hat{x}^i_{i-1}(t))\| \le \frac{\|\hat{x}^i_{i-1}(t) - x^i\|\|x^j\|}{\rho\|x^i\|}.$$

If the network is smooth (has Lipschitz gradient), then interlayer smoothness holds as long as $\|\hat{x}^i_i(t) - x^i\|, \|\hat{x}^i_{i-1}(t) - x^i\|$ is small. Essentially the assumption here is that the network behaves smoothly in the random directions generated by randomly dropping out columns of the matrices.

Similar to (Arora et al., 2018), we have defined multiple quantities measuring the noise stability of a network. These quantities are in practice small constants as we verify experimentally in Section 6. Finally, we combine all these quantities to define a single overall measure of the noise stability of a network.

Definition 4 (Noise Stability)**.** *For a network θ with layer cushion μ_i, minimal interlayer cushion $\mu_{i\to}$, activation contraction c and interlayer smoothness ρ, if the minimum width layer h_{min} is at least $\widetilde{\Omega}(1)$ wide, $\rho \ge 3d$ and $\|\phi(\hat{x}^i_i(t))\|_\infty = O(1/\sqrt{h_i})\|\phi(\hat{x}^i_i(t))\|$ for $1 \le i \le d-1, 0 \le t \le 1$, we say the network θ is ϵ-noise stable for*

$$\epsilon = \frac{\beta c d^{3/2} \max_{x\in S}(\|f_\theta(x)\|)}{h_{\min}^{1/2} \min_{2\le i\le d}(\mu_i\mu_{i\to})}.$$

The smaller ϵ, the more robust the network. Note that the quantity ϵ is small as long as the hidden layer width $h_{\min}$ is large compared to the noise stable parameters. Intuitively, we can think of ϵ as a single parameter that captures the noise stability of the network.

4.2 Noise stability implies dropout stability

We now show that noise stable local minimizers must also be dropout stable, from which it follows that noise stable local minimizers are connected. We first define the dropout procedure we will be using in Algorithm 1.

[2]Note that $J^{i,i}_{x^i} = I_{h_i}$ and $\mu_{i,i} = 1$.

[3]Note that A_1 is excluded because dropping out columns in $\hat{A}_2$ already drops out the neurons in layer 1; dropping out